

Workout generation route Nutrition/Recovery tip generation route

```
# 1. API: Generate workout using Gemini Pro
@router.post("/generate-workout/gemini")
async def generate_gemini_workout(request: WorkoutRequest):
    try:
        result = generate_workout_gemini({
            "goal": request.goal,
            "intensity": request.intensity
        })
        return {"model": "gemini-pro", "workout_plan": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Nutrition/Recovery tip generation route

```
# 2. API: Generate nutrition tip using Gemini Flash
@router.get("/nutrition-tip")
def get_flash_tip(goal: str):
    tip = generate_nutrition_tip_with_flash(goal)
    return {"goal": goal, "nutrition_tip": tip}
```

Saving user info and plan generation route

3. API: Save user info & generate plan

```
@router.post("/generate-plan")
def generate_plan(user_data: UserInput):
    try:
        save_user(
            user_id=user_data.user_id,
            name=user_data.username,
            age=user_data.age,
            weight=user_data.weight,
            goal=user_data.goal,
            intensity=user_data.intensity
        )
        plan = generate_workout_gemini({
            "goal": user_data.goal,
            "intensity": user_data.intensity
        })
        save_plan(user_data.user_id, plan)
        return {
            "message": "Workout plan generated and saved successfully!",
            "workout_plan": plan
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Something went wrong: {str(e)}")
```

Submit Feedback and update plan route

```
# 4. API: Update workout plan based on user feedback
@router.post("/update-plan/{user_id}", response_model=dict)
def update_user_plan(user_id: int, data: FeedbackRequest):
    original = get_original_plan(user_id)
    if not original:
        return {"error": "Original plan not found for this user."}
    updated = update_workout_plan(original, data.feedback)
    update_plan(user_id, updated)
    return {"updated_plan": updated}
```

Milestone 4. Frontend Development

Milestone 4 focuses on developing a user-friendly and visually structured interface for FitBuddy, the AI-powered workout generator. This involves building responsive and interactive web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring users can easily submit input, receive personalized plans, provide feedback, and allow administrators to manage data effectively. This milestone guarantees an engaging and intuitive fitness planning experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

- Developed `index.html` as the main entry point for user input.
- Structured the form to capture:
 - `username`, `user_id`, `age`, `weight`, `goal`, and `intensity`.
- Each field is properly labeled and grouped using semantic HTML.
- The design uses a gym-themed background image, Google Fonts (Roboto), and bold typography to create a clean, fitness-oriented interface.
- Navigation is kept minimal for focus, and user flow directs clearly from input to output.

Design a Responsive Layout Using CSS

- Embedded CSS styles were applied directly in each HTML file.
- Layouts use **Flexbox** to center content both vertically and horizontally.
- Media queries ensure mobile responsiveness.
- Buttons have consistent styling with hover effects.
- Input fields and result blocks have adequate padding, shadows, and rounded corners for modern aesthetics.
- Overall color scheme supports readability on a dark background.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under `/templates`:

- `index.html`: A user input form with fields for fitness goal, intensity, and basic personal information.
- `result.html`: Displays the 7-day AI generated workout plan, a nutrition tip, and accepts user feedback.
- `all_users.html`: Admin panel to view and delete users and review both original and updated plans.

Templates are modular, and UI transitions smoothly between steps.

Forms are submitted to specific FastAPI endpoints via POST methods

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

- Used FastAPI's `Jinja2Templates` for template rendering:

```
from fastapi.templating import Jinja2Templates
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")
templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

- Each route uses `TemplateResponse` to render the proper HTML with passed context: